PA1: Systems Programming, Address Spaces, and Processes

CMPSC 473 – Operating Systems

Introductory / refresher project

1 Goals

This assignment is intended to get you ready for the programming projects in CMPSC 473. It combines a refresher on the systems-programming tools and C concepts that we will assume throughout the semester with an introduction to several operating-system abstractions that we will study in greater depth.

By the end of this assignment, you should be able to:

- reason correctly about C pointers, arrays, dynamic allocation, and object lifetimes;
- compile a multi-file C program using `make` and understand basic Makefile dependencies;
- use `gdb` to investigate program state and diagnose a memory-related failure;
- use Git/GitHub for the basic workflow expected in later projects;
- identify important regions of a process virtual address space and inspect them through `/proc`;
- use `strace` to connect program execution to system calls; and
- observe the effects of `fork` and `exec` on processes and address spaces.

This is deliberately a refresher rather than a comprehensive tutorial on C, Git, Make, or GDB. If some of these tools are unfamiliar, consult their documentation, man pages, and the course references. You are expected to learn enough to complete the tasks below.

2 Getting Started and Repository Rules

This project will be done individually. All subsequent projects will be done in groups of two.

Accept the GitHub Classroom assignment link. This will create a private repository for you. Clone *your repository* onto a Linux machine:

```
git clone <your-repository-url>
cd <repository-directory>
```

You may develop on any suitable Linux system unless otherwise stated. Parts that use `/proc` and `strace` require a Linux environment. If you need access to a department Linux machine, you may use the lab machines in W135. Documentation for lab access is available online at https://www.eecs.psu.edu/cse-student-lab-access/index.aspx and on Canvas.

You may not seek help from another person. You must acknowledge and explain any online resources including AI that you used in doing this project.

Some important notes:

- Do not commit compiled binaries, core dumps, editor backup files, or other generated junk.
- Commit work incrementally. A single commit containing the entire completed assignment does not constitute good use of version control.
- Your repository must build from a clean checkout using the commands specified in this handout.

3 Part I: C Pointers and Memory Refresher

Consider the following program, which will also be provided as `pointer.c`.

```c
#include <stdio.h>
#include <stdlib.h>

struct pair {
    int x;
    int y;
};

static void update(int *p, struct pair *q)
{
    *p += 10;
    q->y = *p;
}

int main(void)
{
    int a[4] = {10, 20, 30, 40};
    int *p = a;
    int **pp = &p;

    struct pair local = {1, 2};
    struct pair *heap = malloc(sizeof *heap);
    if (heap == NULL)
        return 1;

    heap->x = 5;
    heap->y = 6;

    update(&a[1], heap);

    printf("%d %d %d\n", *p, *(*pp + 1), heap->y);
    printf("local = {%d, %d}\n", local.x, local.y);

    /* TODO (Part I): dynamically allocate an array of 100 struct pair
       objects, initialize element i to {i, 2*i}, and free it. */

    free(heap);
    return 0;
}
```

Before running the program, answer the following. Then compile and run it and check your predictions.

1. What are the types of `p`, `pp`, `*pp`, and `**pp`? What objects do they refer to?
2. Explain the difference among `p`, `&p`, `*p`, `a`, and `&a`. You do not need to reproduce formal C type-system terminology, but your explanation must distinguish an address from the object stored at that address.
3. What does `p + 1` mean? If `p` has numeric address A, is the numeric address of `p + 1` normally $A + 1$? Explain.

4. Predict the three integers printed by the first `printf` call.
5. For each of the following, state whether the object itself resides on the stack, heap, or elsewhere: `a`, `p`, `pp`, `local`, `heap`, and the `struct pair` returned by `malloc`. In particular, distinguish the pointer variable `heap` from the object to which it points.
6. Explain why `malloc(sizeof *heap)` is preferable to writing a numeric allocation size. What do `sizeof(heap)` and `sizeof(*heap)` measure?

Next, edit `pointer.c` as follows:

(a) Dynamically allocate an array of 100 `struct pair` objects without hard-coding the size of `struct pair`.

(b) Initialize element i to contain $x = i$ and $y = 2i$ using pointer or array notation.

(c) Free the allocation correctly.

Deliverable: Commit the modified program and include concise answers to Questions 1–6 in your report.

4 Part II: Make and Separate Compilation

The directory `build/` contains three source files:

```
main.c     stats.c     stats.h
```

and an incomplete `Makefile`. Complete the Makefile so that:

- `make` builds an executable named `stats`;
- `main.c` and `stats.c` are compiled separately into object files;
- the dependency on `stats.h` is represented correctly;
- compilation uses at least `-Wall -Wextra -g`; and
- `make clean` removes generated object files and the executable.

After obtaining a successful build, run `make` a second time without modifying any files. Explain why nothing should be recompiled. Then modify only `stats.c` and run `make`; identify what is rebuilt. Finally, touch or modify `stats.h` and explain what should be rebuilt and why.

You should understand the distinction between *compiling* a source file and *linking* object files into an executable. You do not need to use advanced Make features.

Deliverable: Submit the completed Makefile and a short explanation (a few sentences) of the three rebuild experiments.

5 Part III: Debugging with GDB

The program `debug.c` contains a memory bug. Do not begin by randomly editing the program until the crash disappears. Instead, compile it with debugging information and use GDB to understand the failure.

Your investigation must include the following actions:

1. Run the program normally and record the observed failure.
2. Start the program in GDB and place a breakpoint in the function identified in the starter code.

3. Use `next` and/or `step` to follow execution.
4. Use `print` to inspect relevant variables and pointers.
5. Use `x` to examine memory through at least one pointer.
6. When the failure occurs, use `backtrace` and inspect the relevant stack frame.
7. Identify the bug, explain *why* it is invalid C memory behavior, and fix it.

In your report, draw a small memory diagram immediately before the invalid access. Show the relevant pointer(s), the object(s) they point to (or should point to), and whether those objects are on the stack or heap. Numeric addresses from your particular run may be included but are not a substitute for the diagram and explanation.

Deliverable: Submit the corrected source code and a concise debugging narrative containing the bug, evidence from GDB, and memory diagram. Do not submit a transcript of every GDB command.

6 Part IV: Virtual Address Spaces and `/proc`

The program `address-space` recursively invokes a function. Each invocation contains a local variable and performs a small dynamic allocation. At the deepest recursion level the program pauses so that you can inspect it.

Run the program and use the printed addresses together with `/proc/PID/maps` and, where useful, `/proc/PID/smaps` to answer:

1. Which printed addresses correspond to stack objects and which correspond to dynamically allocated heap objects? Explain how you know.
2. In which direction do the stack addresses change as recursion becomes deeper? In which direction do successive heap allocations tend to move in this run? Do not assume that either observation is guaranteed by the C language.
3. Find the virtual-address ranges labeled `[stack]` and `[heap]` in `/proc/PID/maps`. Verify that the printed addresses lie in the region you expect.
4. While the process is paused, report the mapped size of the stack and heap regions. Briefly distinguish a virtual address range from the amount of physical memory actually resident in RAM.
5. Run the program under `strace`. Select **five** system calls that are important or interesting in this execution and explain, in one sentence each, what role they play. You do *not* need to catalog every unique system call.

Useful commands include `pgrep`, `ps`, `cat /proc/PID/maps`, `cat /proc/PID/smaps`, `pmap`, and `strace`. Use the man pages to understand commands and system calls you do not recognize.

Deliverable: Include answers and relevant excerpts from `/proc` in your report. Prefer copied text over screenshots unless a screenshot genuinely communicates something that text cannot.

7 Part V: Memory Errors

The program `memory-bugs.c` contains several independent memory-management mistakes representative of bugs you will encounter in systems code. The starter code labels four test cases. Across the four cases you will encounter examples drawn from the following categories:

- out-of-bounds access,
- use of uninitialized data,
- use after `free`, and
- memory leak.

Compile first with the normal warning flags. Your submission should compile cleanly, but compiler warnings are not intended to reveal the four bugs in this part. For each test case:

1. run the appropriate test normally and record what happens (a program that appears to work may still contain an error);
2. use Valgrind to obtain evidence of the problem;
3. classify the error and identify the responsible source statement(s); and
4. fix the error without changing the intended behavior of the test.

The bugs may be separated from their symptoms by helper functions or runtime-computed sizes. Treat Valgrind's report as evidence, not as a substitute for explaining why the program violates a memory-safety or ownership rule. Do not paste pages of tool output into the report. Quote only the small portion needed to support your diagnosis.

Deliverable: Submit the corrected source and a table with one row per test case: error category, offending statement, one-sentence explanation, and fix.

8 Part VI: Processes, `fork`, and `exec`

The final program consists of `parent.c` and `child.c`. The parent creates a child using `fork`; the child eventually replaces its program using `exec`. The starter program pauses at labeled points so that you can inspect both processes without relying on fragile source-code line numbers.

Answer the following:

1. Immediately after `fork`, compare the parent and child PIDs and the virtual addresses of a global variable, a stack variable, and a dynamically allocated object. Some virtual addresses may be equal. Does equal virtual address imply that the parent and child are using the same physical memory location? Explain at a high level.
2. The starter code already modifies observed variables in the child after `fork`. Does the corresponding value in the parent change? Explain the result in terms of process address spaces and copy-on-write.
3. Compare `/proc/PID/maps` for the child immediately before and after successful `exec`. What changed? What happened to the old program's code, data, heap, and stack?
4. Use `strace -f` (or an equivalent option you discover from the man page) to follow both processes. Identify the `fork`/`clone`-related call, the `exec` call, and the parent's wait-related call.

You may use GDB's `set follow-fork-mode` and `set detach-on-fork` if useful, but the assignment does not require an elaborate GDB session involving both processes.

Deliverable: Include concise answers and the relevant evidence. Do not submit screenshots merely to prove that a command was run.

9 Part VII: Git/GitHub Workflow

Version control is part of the assignment rather than merely the submission mechanism. During the project, your repository should contain a sensible sequence of commits. At minimum, complete the following workflow:

1. After completing Part I, commit it with a descriptive message.
2. Create a branch named `make-refresher`, complete the Makefile work there, commit it, and merge the branch into your main branch.
3. Complete the remaining parts using incremental commits rather than one final bulk commit.
4. Before submission, run `git status` and make sure all intended source/report files are tracked and that generated files are not accidentally committed.

You should be comfortable with `git status`, `git diff`, `git add`, `git commit`, `git log`, `git branch`, `git switch` (or `git checkout`), `git merge`, `git pull`, and `git push`.

Commit count alone is not a measure of quality; dozens of meaningless commits are not useful. Prefer a small number of clear, descriptive commits that reflect completed work.

Deliverable: No separate Git report is required. The repository history itself is the deliverable.

10 Submission

Push the following to your GitHub Classroom repository before the deadline:

- all required C source and header files;
- your completed Makefiles;
- a `README.md` containing any non-obvious build/run instructions; and
- a report named `report.pdf` answering the questions above.

A grader should be able to clone the repository and build the programs from source. Unless a part explicitly asks for raw output, your report should contain explanations and selected evidence rather than long command transcripts or screenshots.

11 Grading

Component	Weight
C pointers and memory refresher	15%
Make and separate compilation	10%
GDB debugging	15%
Address spaces and `/proc/strace`	20%
Memory errors	15%
`fork`/`exec` and process address spaces	20%
Git workflow / repository quality	5%

The emphasis is on understanding, not clerical work. Correct code without an adequate explanation may receive only partial credit on questions asking for reasoning; similarly, a polished report cannot compensate for code that does not build or run.

12 Compact Tool Reference

This section is a reminder, not a substitute for documentation.

GDB

```
gdb ./program
break function_name
run
next                # execute next source line, stepping over calls
step                # step into a function call
print expression
x/16gx address      # example: examine memory
backtrace
frame N
info locals
continue
quit
```

Process and address-space inspection

```
pgrep program
ps -ef | grep program
cat /proc/PID/maps
cat /proc/PID/smaps
cat /proc/PID/limits
pmap PID
strace ./program
strace -f ./program
```

Git

```
git status
git diff
git add <files>
git commit -m "descriptive message"
git log --oneline
git switch -c branch-name
git switch main
git merge branch-name
git pull
git push
```

Make

```
make
make clean
make -n              # show commands without executing them
```

A Note on What Was Intentionally Left Out

This assignment does not ask you to repeatedly compare 32-bit and 64-bit executable/library sizes, enumerate every system call emitted by a program, manually estimate the maximum recursion depth from stack-frame sizes, or collect large numbers of screenshots. Those activities can consume substantial time without proportionate conceptual benefit. Instead, the project focuses on skills and observations that will recur in later operating-systems work: pointers and memory safety, building and debugging C programs, virtual address spaces, system calls, and process creation/replacement.